ELEC3104: Digital Signal Processing

Topic 1: DSP fundamentals
MATLAB Examples

Prof. Eliathamby Ambikairajah, School of EE&T

Term 3, 2026

Topic 1: DSP fundamentals: Discrete-time sinusoidal signal

Example 1

A discrete-time sinusoidal signal is given by:

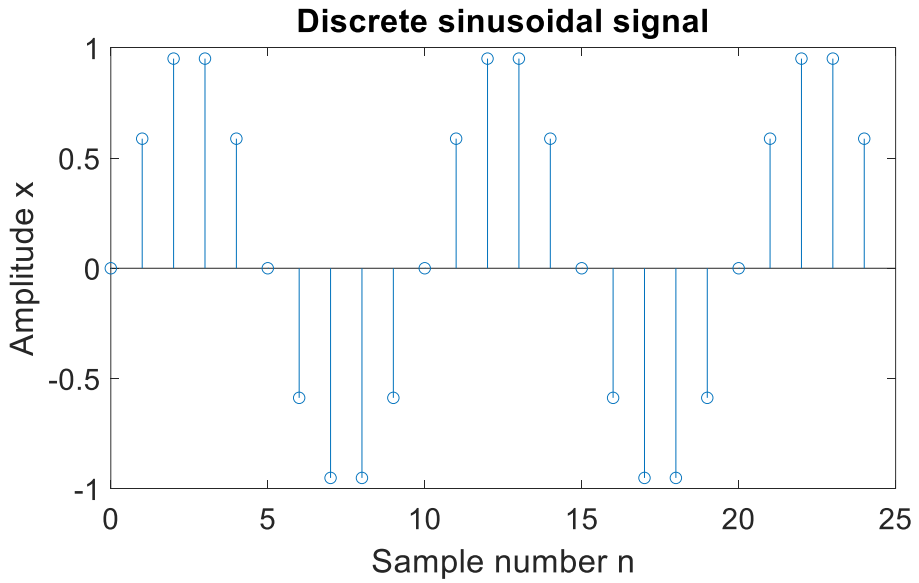
$$x[n] = A\sin\left(2\pi\frac{f_a}{f_s}\mathrm{n}\right)$$

Where the analog frequency $(f_a) = 100\ Hz$, the sampling frequency $(fs) = 1\ kHz$, and $n = 0,1,2,\ldots\ldots 24$.

a. Write a MATLAB code to generate and plot $x[n]$

b. Using the plot of $x[n]$ obtained in part (a) above, count the number of samples (N) in one period and verify it using the equation $\mathrm{N} = \frac{f_s}{f_s}k$ where k is a positive integer.

```
% Discrete-time sinusoidal signal generation
n = 0:24; % n goes from 0 to 24
fa = 100; % analog frequency in Hz
fs = 1000; % sampling frequency in Hz
x = sin(2*pi*n*fa/fs); % x = [x1, x2,.....,x25]
figure % create figure window
stem(n,x) % stem plot
xlabel('Sample number n') % labeling the x-axis
ylabel('Amplitude x') % labeling the y-axis
title('Discrete sinusoidal signal') % title of the figure
```



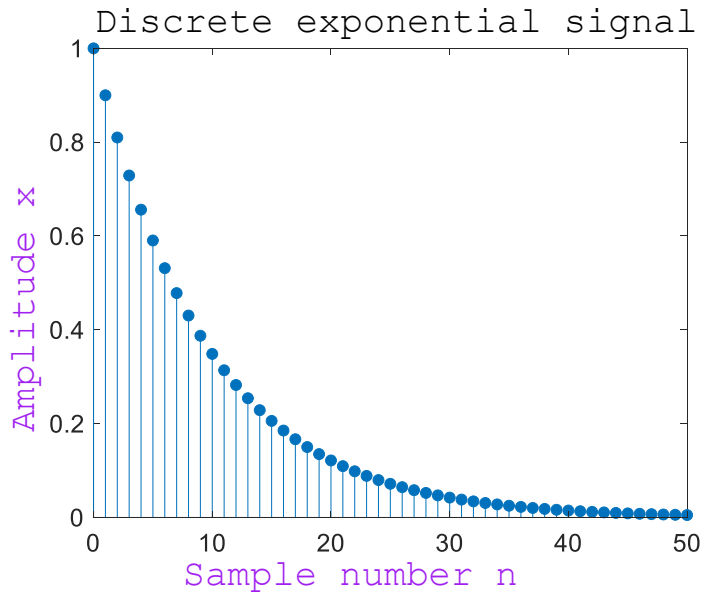
Topic 1: DSP Fundamentals - Discrete-time signals

Example 2

Write a MATLAB code to generate and plot the signal

$x[n] = 0.9^n$, where $0 \leq n \leq 50$.

```
% Exponential signal generation
n = 0:50;
x = 0.9.^n;
figure
stem(n, x, 'filled') % stem circles filled
xlabel('Sample number n')
ylabel('Amplitude x')
title('Discrete exponential signal')
```

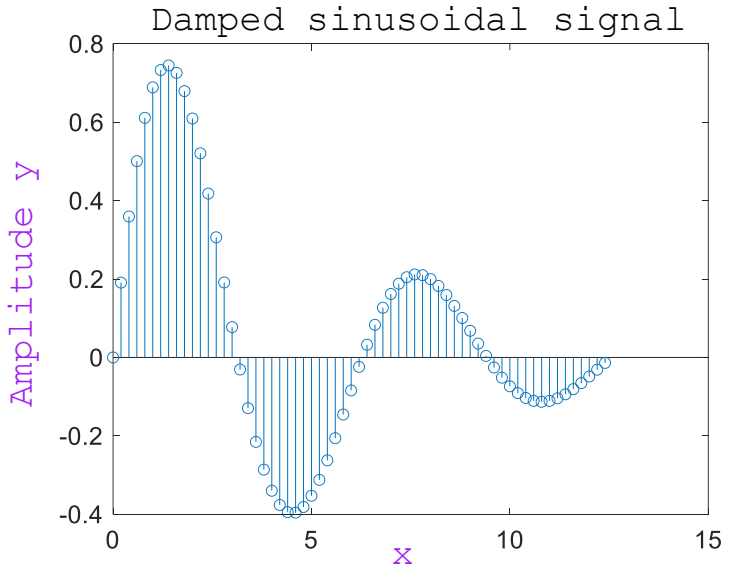


Example 3

Write a MATLAB code to generate and plot the signal

$y[x] = e^{-0.2x}\ \sin(x)$, where $x = 0, 0.2, 04, \ldots, 4\pi$

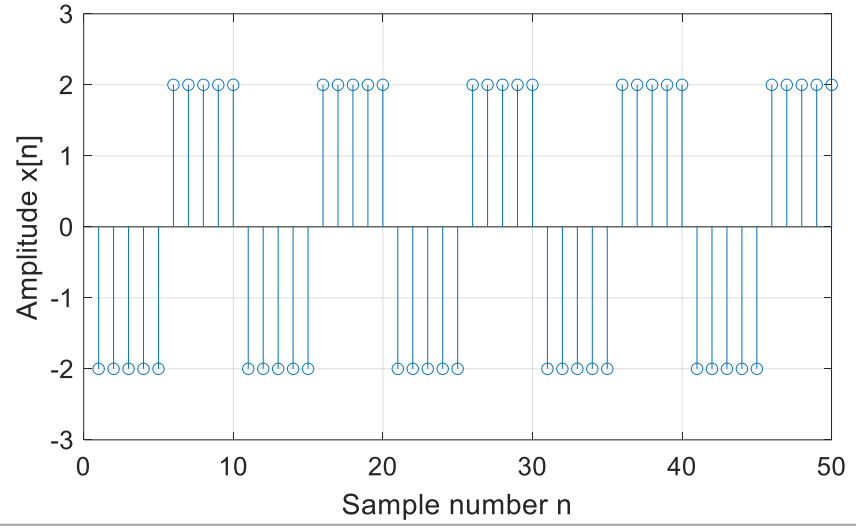
```
% Damped sinusoidal signal generation
x = 0:0.2:4*pi; % x = [0, 0.2, 0.4, ....., 4π]
y = exp(-0.2*x).*sin(x);
figure
stem(x, y)
xlabel('x')
ylabel('Amplitude y')
title('Damped sinusoidal signal')
```



Topic 1: DSP Fundamentals - Discrete-time signals

Example 4

Write a MATLAB code to generate the following signal x[n].




```
% rectangular pulse signal generation
% x = 2*[-1,-1,-1,-1,-1,1,1,1,1,1]
% alternatively, "ones" array can be used
x = 2*[-ones(1,5), ones(1,5)];
y = [x, x, x, x, x]; % Repeat 5 times
stem(y)
% set axis limits such that x ranges
% from 0 to 50 and y from -3 to 3
axis([0 50 -3 3])
grid on; % Display the grid lines
```

Example 5

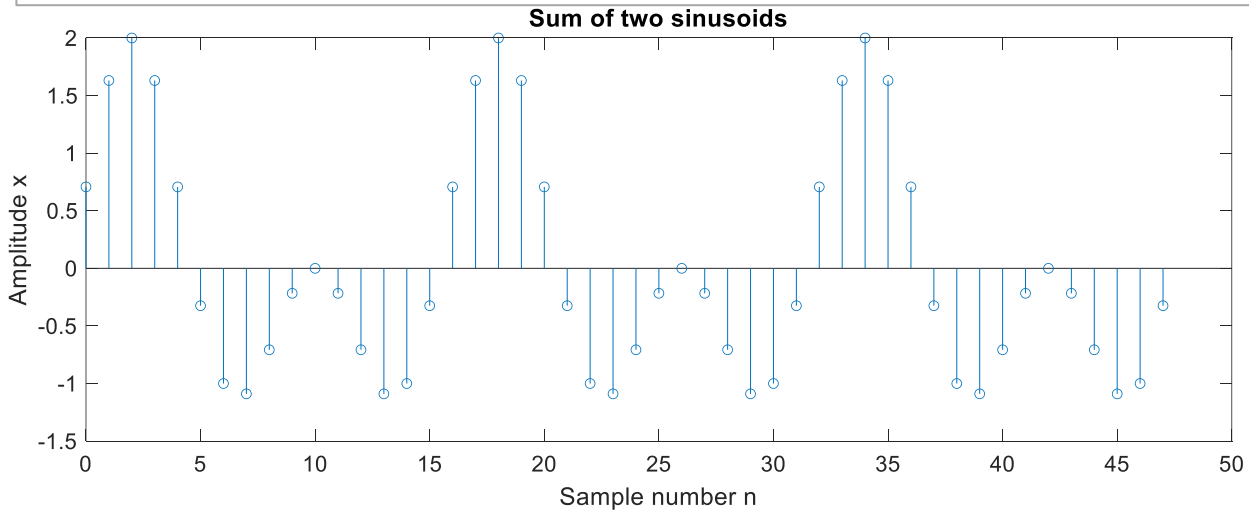
Consider the discrete-time sequence $x[n] = \sin(\frac{n\pi}{4}) + \sin(\frac{n\pi}{8} + \frac{\pi}{4})$

a. Write a MATLAB code to generate 48 samples of $x[n]$ and plot it.

b. Using the plot of $x[n]$, count the number of samples (N) in one period and verify it using the equation $N = LCM(N1, N2)$, where $N1$ and $N2$ are the periods of $\sin(\frac{n\pi}{4})$ and $\sin(\frac{n\pi}{8} + \frac{\pi}{4})$ respectively.

LCM – Least Common Multiple

```
% Sum of two sinusoidal signal
n = 0:47;
x = sin(0.25*n*pi) + sin(0.125*n*pi + pi/4);
stem(n, x);
xlabel('Sample number n');  ylabel('Amplitude x')
title('Sum of two sinusoids')
```



Topic 1: DSP Fundamentals - Frequency Domain Analysis using FFT

Example 6

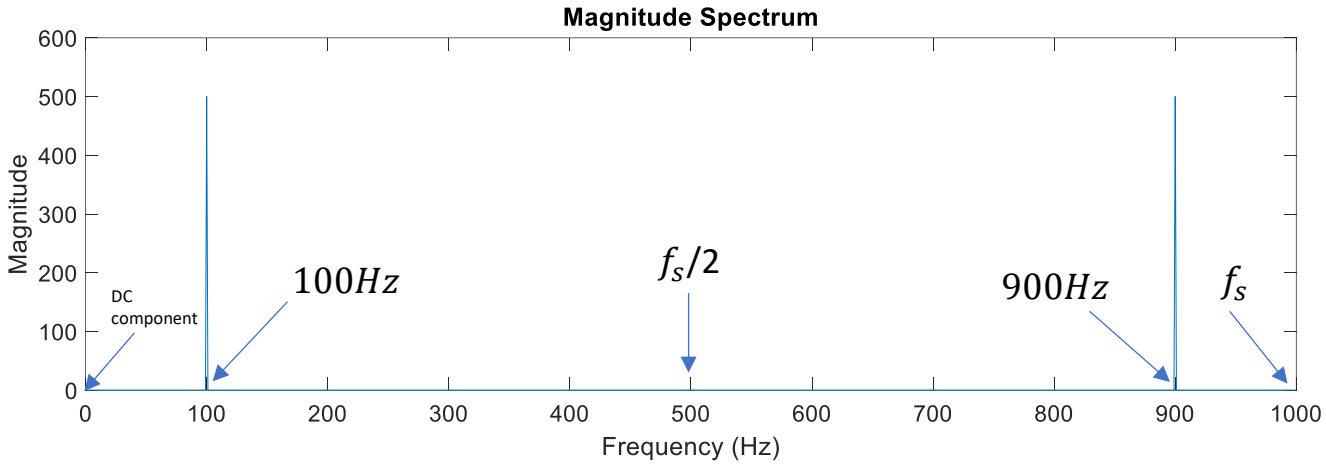
a. Write a MATLAB code to generate 1000 samples of

$$x[n] = sin(\frac{2\pi n f_a}{f_s})$$

where $f_a = 100Hz, f_s = 1\text{k}Hz, n = 0\ to\ 999.$

b. Using Fast Fourier Transform (FFT) { a computationally efficient method of computing the Fourier transform of a discrete-time signal}, compute and plot the magnitude spectrum of $x[n]$ to verify that frequency $f_a = 100\ Hz$ is present in the spectrum.

```
% Calculating the magnitude spectrum of a signal using FFT
fa = 100; % Signal frequency in Hz
fs = 10^3; % Sampling frequency in Hz
n = 0:999; % 1000 samples
x = sin(2*pi*n*fa/fs); % discrete sinusoid
len = length(n); % length of the signal
y = fft(x, len); % FFT produces complex spectrum y
mag_y = abs(y); % the magnitude spectrum
resolution = fs/len; % in Hz
frequency_x = (0:len-1)*resolution; % frequency
figure
plot(frequency_x, mag_y);
xlabel('Frequency (Hz)');
ylabel('Magnitude');
title('Magnitude Spectrum');
```

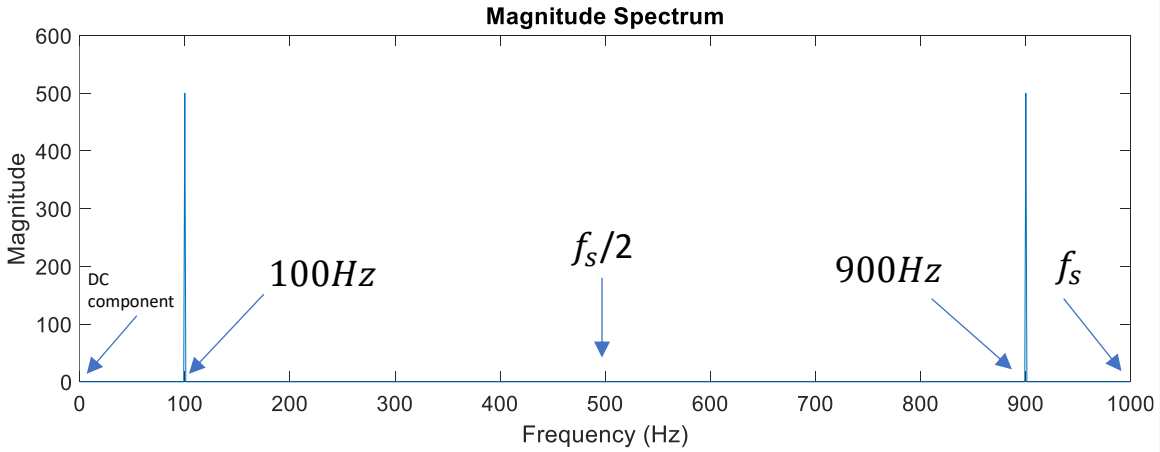


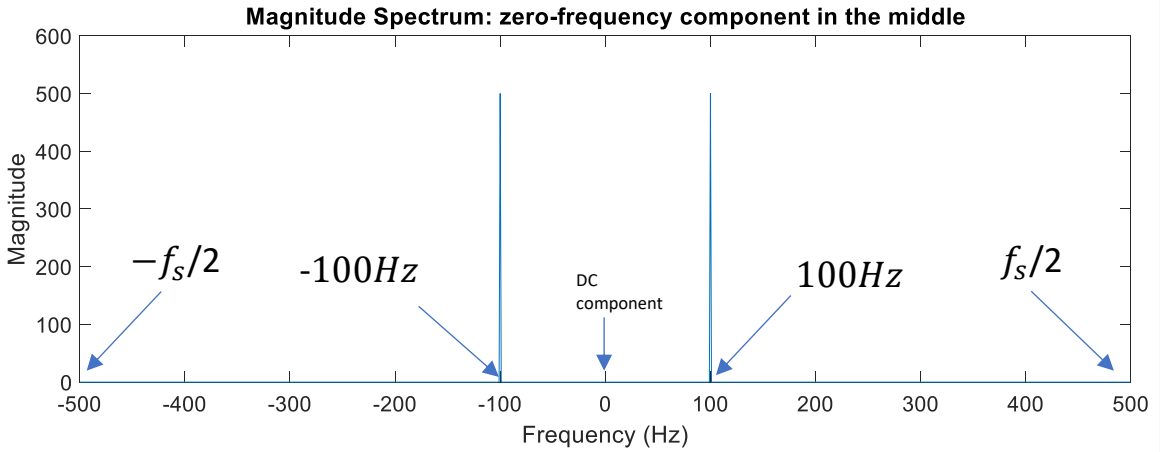
Discrete Fourier Transform (DFT)/Fast Fourier Transform (FFT)

- Number of time samples = Number of frequency samples (k) = N
- Let f_s = 1000 Hz, let the number of samples N = 1000
- $\therefore$ Frequency resolution $\Delta f = \frac{f_s}{N} = \frac{1000}{1000} = 1Hz$
- $f_k = k\left(\frac{f_s}{N}\right) = k\Delta f \quad (k = 0, 1, 2, \ldots, N-1)$
- $f_0 = 0;\ f_1 = 1\text{Hz},\ f_2 = 2\ \text{Hz},\ \ldots.\ f_{999} = 999\text{Hz}$
- The magnitude of the FFT (includes a scaling factor) is proportional to the amplitude of the associated frequency component present in the signal.
- Magnitude Spectrum is symmetrical, if the input signal is real.
- The DC component (f_0) of this signal is zero.

FFT shift command

- It is useful for visualizing a Fourier transform with the zero-frequency component (DC component) in the middle of the spectrum.






```
% Calculating the magnitude spectrum of the signal using "shiftfft"
fa = 100; % Signal frequency in Hz
fs = 10^3; % Sampling frequency in Hz
n = 0:999; % 1000 samples
x = sin(2*pi*n*fa/fs); % discrete sinusoid
len = length(n); % length of the signal
y = fft(x, len); % FFT produces complex spectrum y
mag_y = abs(y);
resolution = fs/len;
frequency_x = (0:len-1)*resolution; % 0 Hz to fs Hz

% FFT shift : To visualize the zero-frequency component in the middle of the
spectrum
y_shift = fftshift(y);
mag_y_shift = abs(y_shift);
fshift = (-len/2:len/2-1)*resolution; % -fs/2 Hz to +fs/2 Hz

figure
% breaks the figure into subplots
subplot 211 % 2 rows, 1 column and first figure
plot(frequency_x, mag_y);
xlabel('Frequency (Hz)'); ylabel('Magnitude'); title('Magnitude Spectrum');
ylim([0 600]); % set the y-axis limit

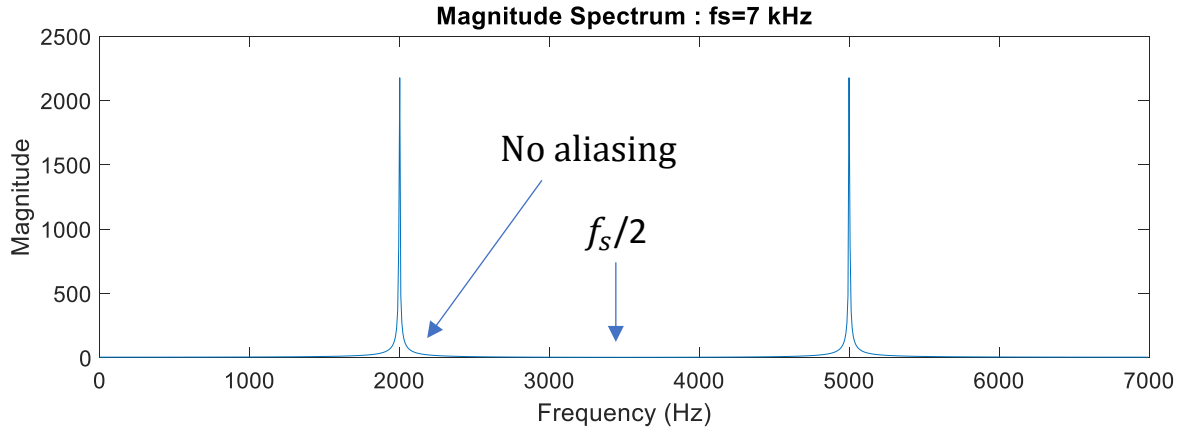
Subplot 212 % 2 rows, 1 column and second figure
plot(fshift, mag_y_shift);
xlabel('Frequency (Hz)'); ylabel('Magnitude');
title('Magnitude Spectrum: zero-frequency component in the middle');
ylim([0 600]);
```

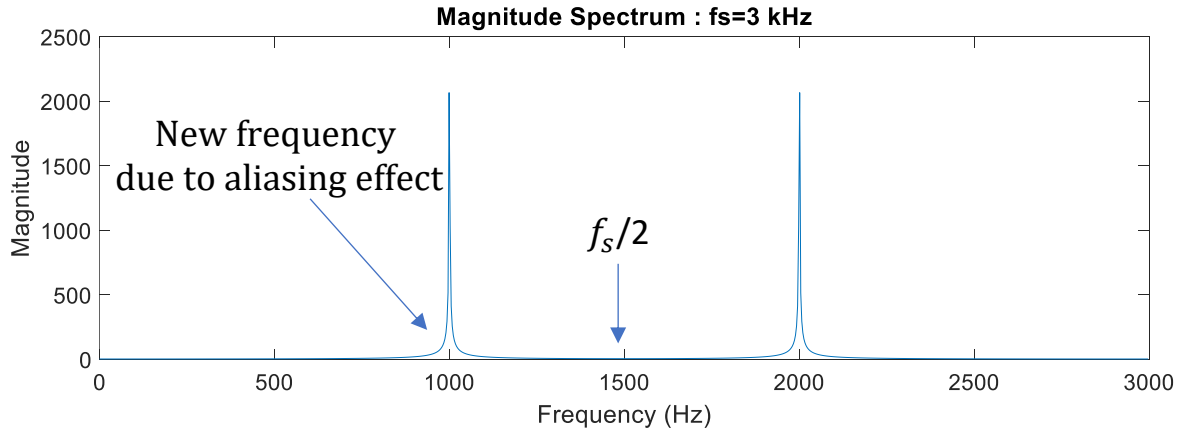
Topic 1: DSP Fundamentals - Sampling Theorem and Aliasing

Example 7

An analogue signal $x(t) = 5\sin(4000\pi t + \pi/4)$ is sampled at a rate of f_s to obtain the discrete-time signal $x[n]$.

Write a MATLAB code to generate 1000 samples of $x[n]$ and plot the magnitude spectrum of $x[n]$ in the frequency region from 0 to fs for the following sampling frequencies: (a) $f_s = 7\ kHz$, (b) $f_s = 3\ kHz$





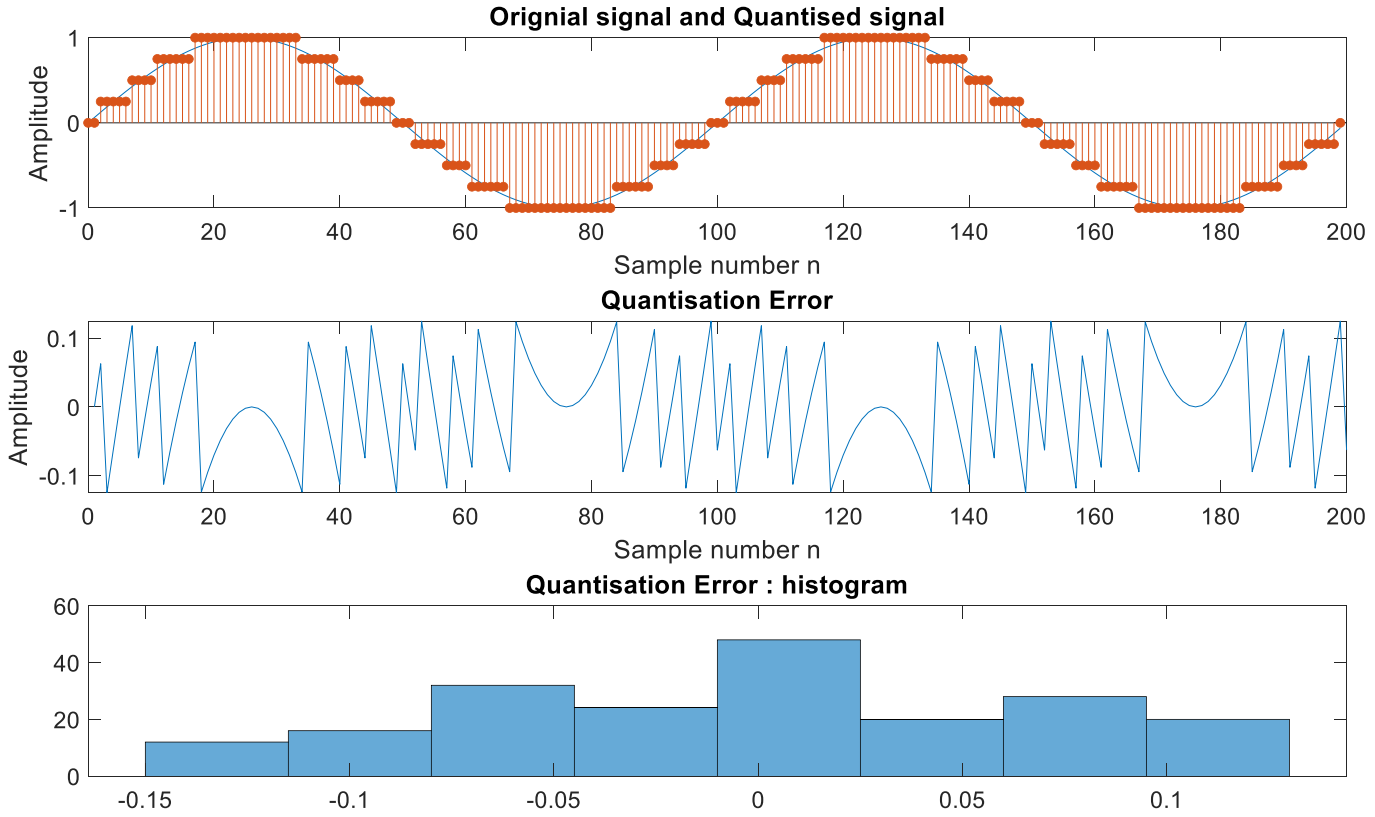

```
% (a) Sampling frequency 7000 Hz
fs = 7000;
% Analog frequency of the sinwvae 2000 Hz
fa = 2000; % fs > 2*fa hence no aliasing
n = 0:999; % 1000 samples
len = length(n); % length of the signal
x = 5*sin(2*pi*n*fa/fs + pi/4); % discrete sinusoid
freqHz = (0:len-1)*(fs/len); % 0 Hz to fs Hz
y = fft(x, len); % FFT produces complex spectrum y
figure
subplot 211 % 2 rows, 1 column and first figure
plot(freqHz, abs(y));
xlabel('Frequency (Hz)'); ylabel('Magnitude');
title(['Magnitude Spectrum: fs = ' num2str(fs*10^-3) ' kHz']);

% (b) Sampling frequency 3000 Hz
fs = 3000; % fs < 2*fa hence aliasing occurs
x = 5*sin(2*pi*n*fa/fs + pi/4); % discrete sinusoid
freqHz = (0:len-1)*(fs/len); % 0 Hz to fs Hz
y = fft(x, len); % FFT produces complex spectrum y
subplot 212 % 2 rows, 1 column and second figure
plot(freqHz, abs(y));
xlabel('Frequency (Hz)'); ylabel('Magnitude');
title(['Magnitude Spectrum: fs = ' num2str(fs*10^-3) ' kHz']);
```

Example 8: Signal Quantisation and SQNR (Page 30)

The signal $x(t) = \sin(2\pi t)$ is sampled at $100\ Hz$ and 200 samples are obtained starting at $t = 0$. The sampled signal $x[n]$ is quantised by rounding using a 3-bit quantiser to obtain the new quantised signal $x_Q[n]$

a. Plot the signal $x(t)$ and the quantised signal $x_Q[n]$.
b. Plot the histogram of the error signal $x[n] - x_Q[n]$.
c. Find the quantisation SQNR.
d. Find the SQNR using the expression valid for sinusoids (SQNR = 6*B + 1.76) where B is the number of bits. Compare the results that you obtained with part (c) above.
e. Re-run the code for different B values and discuss your results.




```
% Signal to quantisation noise ratio (SQNR)
fs = 100; % sampling frequency 100 Hz
fa = 1; % Analog frequency of the sinwvae 1 Hz
n = 0:199; % 200 samples
x = sin(2*pi*n*fa/fs); % discrete sampled signal (t = n/fs)
R = max(x) - min (x); % dynamic range
B = 3; % number of bits
L = 2^B; % number of levels
resolution = R/L;
% Qunatised signal by rounding
y = resolution * round(x/resolution);
% use the following for truncation
% y = resolution * floor(x/resolution);
figure
subplot 311 % three plots
plot(n,x); % plot the original signal
title('Original signal and Quantised signal');
hold on % hold the figure
stem(n, y, 'filled'); % stem plot of quantised signal
hold off
err = x - y; % error signal
subplot 312
plot(err);
title('Quantised Error');
subplot 313
% error is uniformly distributed
histogram(err, L); % histogram
title('Quantised Error: histogram');

% Estimate signal to noise quantised ratio
% element-wise multiplication
sqnr = 10*log10(sum(x.*x)/sum(err.*err));
% formula-based calculation
% (see lecture notes page 30)
sqnr_formula = 6 * B + 1.76;
```

Topic 1: DSP Fundamentals: Time-Frequency Domain Analysis

Example 9: Time-varying stepped frequency

A simple example of time-varying frequency content is to make a waveform whose frequency holds for a short duration and then steps to a higher frequency. For example , C-major scale consists of playing the notes [C(262Hz), D(294Hz), E(330Hz), F(349Hz), G(392Hz), A(440Hz), B(494Hz), C(524Hz)].

a. Write a MATLAB code to generate the signal with the frequency 262Hz for 200ms, then the frequency 294Hz during the next 200ms and so on. The total waveform duration consisting the notes {C,D,E,F,G,A,B,C} will be 1.6 sec. You may choose the sampling frequency as 8000Hz.
b. Play the signal that you have generated using the 'sound' command. Make sure you insert silence between the notes
c. Now, play the notes again in the following format : {C,D,E,F,G,A,B,C,B,A,G,F,E,D,C}

```
% Playing the musical notes
fs = 8000; % Sampling frequency
d_ms = 200; % duration of each note in ms
num_samples = d_ms*10^-3*fs;
n = 0:num_samples-1; % total number of samples in 200 ms
freq_notes = [262,294,330,349,392,440,494,524]; % in Hz

x = [];
% n number of 0s for generating silence between notes
gap = zeros(1, 1.5*num_samples); % 300 ms gap

for i=1:length(freq_notes) % number of notes
% each note frequency samples are cascaded in one array
silance_appended = [sin(2*pi*n*freq_notes(i)/fs) gap];
x = [x silance_appended];
end
% Part b: play the generated musical notes
sound(x, fs);

% Part c: Ascending and Descending
flip_x = flip(x);
A_D_sound = [x flip_x(2:end)];
sound(A_D_sound, fs); % play the sound
```

ELEC3104: Digital Signal Processing

Topic 1: DSP fundamentals
MATLAB Exercises

Prof. Eliathamby Ambikairajah, School of EE&T

Term 3, 2026

Exercises - DSP Fundamentals (Topic 1)

Exercise 1

An analogue signal $x(t) = 3\cos(4000\pi t) + 5\sin(6000\pi t) + 10\cos(12000\pi t)$ is sampled at a rate of f_s to obtain the discrete-time signal $x[n]$.

Write a MATLAB code to generate 1000 samples of $x[n]$ and plot the magnitude spectrum of $x[n]$ in the frequency region from 0 to fs for each of the following sampling frequencies: (a) $f_s = 15\ kHz$, (b) $f_s = 10\ kHz$: (c) $f_s = 7\ kHz$, (d) $f_s = 5\ kHz$

Discuss your code & the results with your lab demonstrator.

Exercise 2

An amplitude modulated signal s[n] is given by $s[n] = [1 + m\ sin(2\pi n \frac{f_m}{f_s})]\cos(2\pi n \frac{f_c}{f_s})$, where the modulation index m = 0.9, modulation frequency f_m=100 Hz, carrier frequency f_c=1 kHz, sampling frequency f_s=10 kHz, sample number n = 0, 1, 2, …, 511.

a. Write a MATLAB program to generate and plot s[n].
b. Plot the magnitude spectrum of s[n] in the frequency region [-f_s/2, f_s/2] Hz and identify the frequency components present in the signal.

Exercise 3

A noisy signal is given by $x[n] = s[n] + A\ \varepsilon[n]$ with a signal component $s[n] = \sin(2\pi \frac{f_a}{f_s})$ (where analog frequency f_a = 1 Hz, sampling frequency f_s = 100 Hz) and the noise component $A\ \varepsilon[n]$ .The signal to noise ratio (SNR) of the signal power σ_s^2 to noise power $A^2\sigma_n^2$ is defined in decibels (dB) as $SNR = 10\ log \frac{\sigma_s^2}{A^2\sigma_n^2} dB$. We can adjust the SNR by varying the noise amplitude A. Using the above equation, write a MATLAB code to:

a. Generate 200 samples of $x[n]$ and plot the noisy sinusoid with a signal-to-noise ratio of 5 dB. (Gaussian noise can be generated using "randn" function, standard deviation can be calculated using the command "std" in MATLAB)
b. Plot the magnitude spectrum of $x[n]$ for SNR = 5 dB. Vary the SNR from 0 dB to 20 dB and discuss your results with your lab demonstrator.

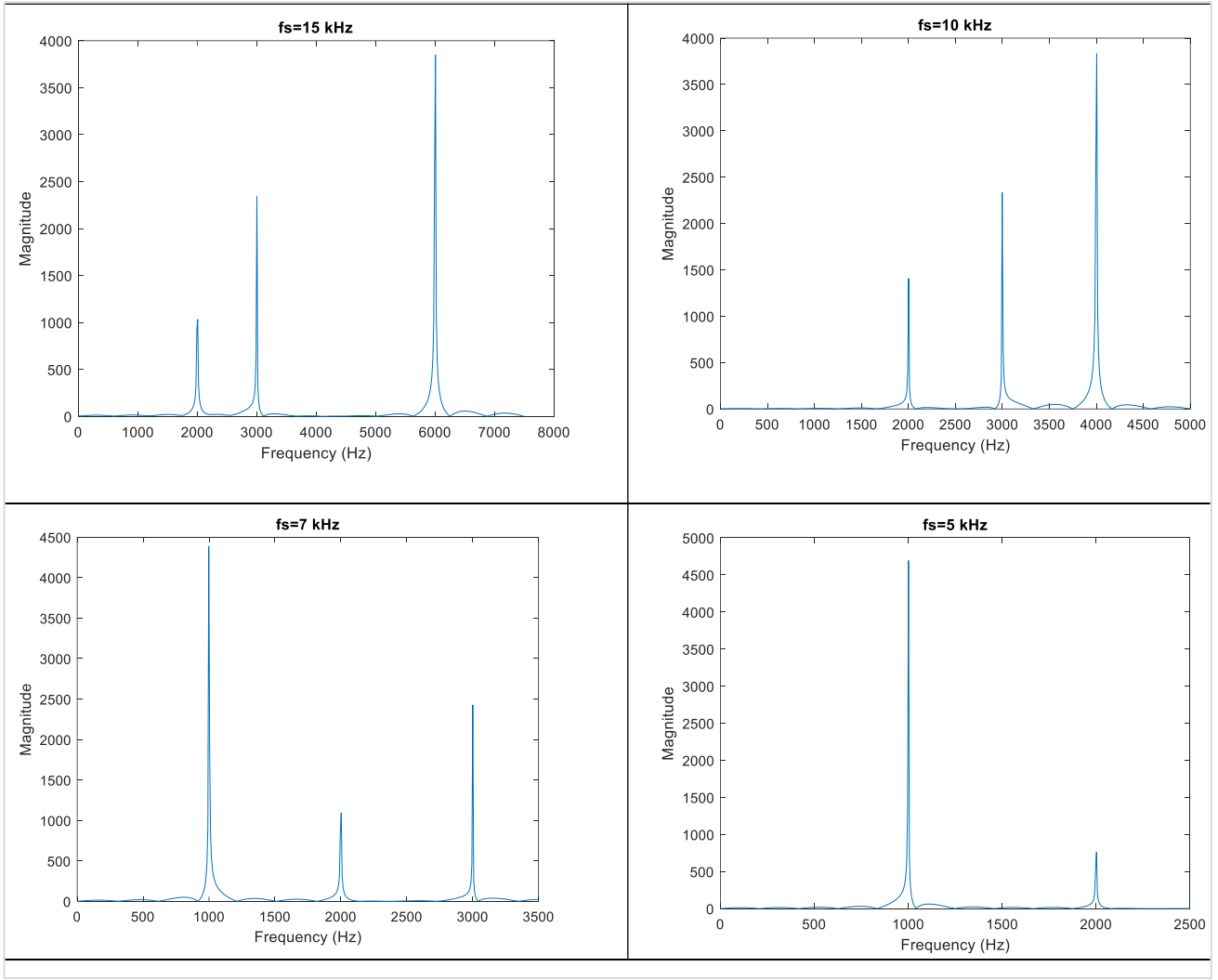
Exercises - DSP Fundamentals (Topic 1)

Exercise 1

An analogue signal $x(t) = 3\cos(4000\pi t) + 5\sin(6000\pi t) + 10\cos(12000\pi t)$ is sampled at a sampling frequency of f_s to obtain the discrete-time signal $x[n]$.

Write a MATLAB code to generate 1000 samples of $x[n]$ and plot the magnitude spectrum of $x[n]$ in the frequency region from 0 to f_s for each of the following sampling frequencies:
(a) $f_s = 15\ kHz$, (b) $f_s = 10\ kHz$: (c) $f_s = 7\ kHz$, (d) $f_s = 5\ kHz$

```
% Sampling frequencies
fs_values = [15000 10000 7000 5000];
for k = 1:length(fs_values)
        fs = fs_values(k);
        f1=2000; f2=3000; f3=6000;
        n = 0:999;
        N = 1024; % number of frequency points when compute fft
        x=3*cos(2*pi*n*f1/fs)+5*sin(2*pi*n*f2/fs)+10*cos(2*pi*n*f3/fs);
        resolution = fs/N;
        freqHz = (0:N-1)*resolution; % Frequency axis: 0 to fs
        freqHz = freqHz(1:N/2); % Frequency axis: 0 to fs/2
        X = fft(x, N); % spectrum in frequency region [0, fs]
        X = X(1:N/2); % spectrum in frequency region [0, fs/2]
        X_mag = abs(X);
        figure;
        plot(freqHz, X_mag);
        xlabel('Frequency (Hz)'); ylabel('Magnitude');
        title(['Magnitude Spectrum, fs = ', num2str(fs*10^-3), ' kHz']);
end
```



Exercises - DSP Fundamentals (Topic 1)

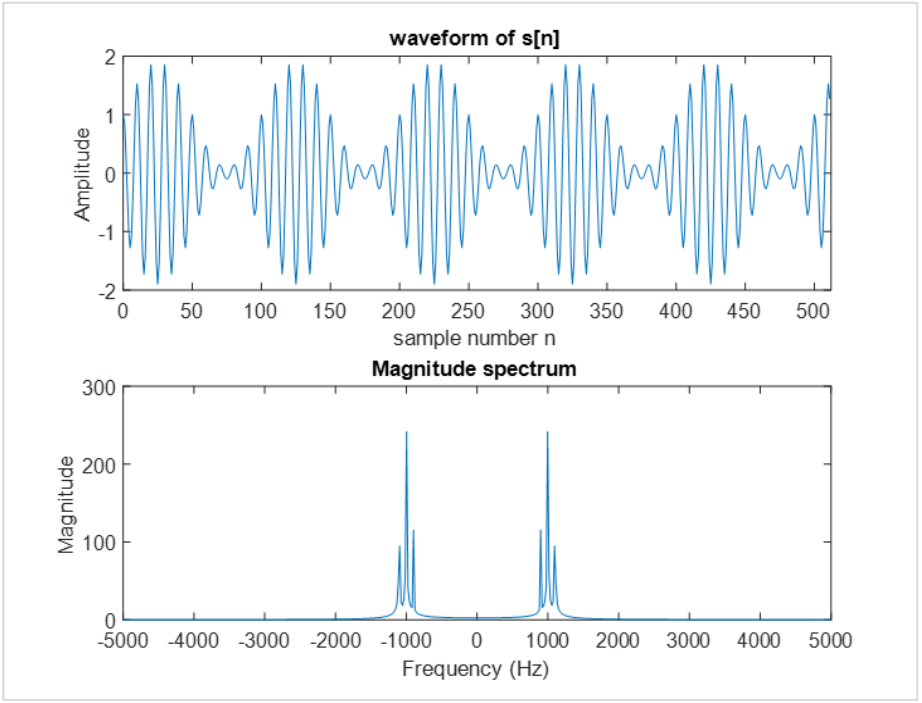
Exercise 2

An amplitude modulated signal s[n] is given by $s[n] = [1 + m\,sin(2\pi n\frac{f_m}{f_s})]\cos(2\pi n\frac{f_c}{f_s})$, where the modulation index m = 0.9, modulation frequency f_m=100 Hz, carrier frequency f_c=1 kHz, sampling frequency f_s=10 kHz, sample number n = 0, 1, 2, …, 511.

a. Write a MATLAB program to generate and plot s[n].

a. Plot the magnitude spectrum of s[n] in the frequency region [-f_s/2, f_s/2] Hz and identify the frequency components present in the signal.

```
m = 0.9; fm=100; fc=10^3; fs=10^4;
n=0:511;
len = length(n);
s=(1+m*sin(2*pi*n*fm/fs)).*cos(2*pi*n*fc/fs);
y=fft(s,len); % manitude spectrum in [0, fs] Hz
% To visualize the the zero-frequency component in the middle
y_shift = fftshift(y);
fshift = (-len/2 : len/2 -1)* (fs/len);
figure
subplot 211
plot(n, s)
xlabel('sample number n');  ylabel('Amplitude');  title('waveform of s[n]');
xlim([0 512]);
subplot 212
plot(fshift, abs(y_shift));
ylabel('Magnitude');  xlabel('Frequency (Hz)');  title('Magnitude spectrum');
```



Exercises - DSP Fundamentals (Topic 1)

Exercise 3

A noisy signal is given by $x[n] = s[n] + A\,\varepsilon[n]$, where $s[n] = \sin(2\pi \frac{f_a}{f_s} n)$ with analogue frequency f_a = 1 Hz and sampling frequency f_s = 100 Hz. The noise sequence ε[n] is zero-mean Gaussian noise and A controls the noise amplitude. The signal to noise ratio (SNR) of the signal power σ_s^2 to noise power $A^2\sigma_\epsilon^2$ is defined as $SNR = 10\,log_{10}\frac{\sigma_s^2}{\sigma_\epsilon^2} dB$. We can adjust the SNR by varying the noise amplitude A. Using the above equation, Write a MATLAB code to:

a. Generate 200 samples of x[n] and plot the noisy sinusoidal signal for an SNR of 5 dB. Gaussian noise may be generated using the MATLAB function *randn*, and the standard deviation may be calculated using *std*.
b. Plot the magnitude spectrum of $x[n]$ for SNR = 5 dB.
c. Vary the SNR from 0 dB to 20 dB and discuss the effect on the time-domain signal and its magnitude spectrum.

```
t=0:0.01:2; % time array - 201 samples
s = sin(2*pi*t); % sinusoidal singal
epsilon=randn(size(s)); % Gaussian noise
snr=5; % signal to noise ratio
% compute noise amplitude
A = std(s)/(std(epsilon)*sqrt(10^(snr/10)));
x = s+A*epsilon; % generate noisy signal
y = abs(fft(x));
figure
subplot 211
plot(x);
xlabel('Sample number');  ylabel('Amplitude');  xlim([0,205]);
subplot 212
plot(y);
xlabel('Sample number');  ylabel('Magnitude');  xlim([0,205]);
```

